

QUARTZ: Thermodynamic Value-of-Computation Control for Neural Monte Carlo Tree Search

A CPU-centric framework with Nested Sampling, One-loop Corrections, Free-energy Modulation, and Discrete Curvature

Jiwon Park

January 29, 2026

Abstract

We formalize a research program to build a game-agnostic, consumer-CPU-friendly board-game AI that exceeds the standard AlphaZero paradigm by replacing fixed-budget Monte Carlo Tree Search (MCTS) with *dynamic computation allocation*. The key idea is to cast MCTS as a *rational metareasoning* problem: at any moment the agent chooses a *compute action*—refine an already-considered move, widen the move set, reconcile inconsistent estimates, or stop—to maximize expected improvement of the *root decision* per unit cost. We introduce QUARTZ, a unified framework that (i) interprets policy-guided selection as a finite-temperature (thermodynamic) decision rule and adds a principled one-loop correction to exploration, (ii) activates Nested Sampling-style exploration only when rollout noise or priors indicate heavy tails or multimodality, (iii) uses free-energy / surprise as a hyperparameter-light modulator of expected improvement and a gate for rare-mode search, and (iv) quantifies search difficulty via discrete curvature defined either on the transposition graph (holonomy residuals) or intrinsically on the tree (graph Ricci curvature) when transpositions are sparse. We provide a full mathematical specification, state explicit assumptions, prove core propositions (variational free-energy equivalence, Gaussian flip probability, termination under diminishing value of computation, and calibrated-halting error bounds), and present complete pseudocode. A rigorous `SearchController` API is specified to enable spec-driven development and fair baseline comparisons.

Contents

1	Problem Setting and Design Constraints	3
1.1	Game model	3
1.2	Neural prior and value	3
1.3	Compute constraints	3
1.4	Baseline fairness contract	3
2	Preliminaries: MCTS, PUCT, and Parallelism	4
2.1	Edge statistics	4
2.2	PUCT selection	4
2.3	Progressive widening	4
3	Unified Latent-Utility Belief Model	4
3.1	Latent utility and observations	4
3.2	Posterior sketch and robust moments	4

4	Thermodynamic View: Finite Temperature and Free Energy	5
4.1	Variational free-energy form	5
4.2	One-loop correction as a cumulant term	5
5	Rational Metareasoning and Value of Computation	5
5.1	Compute actions	5
5.2	Value of computation (VOC)	6
5.3	Global VOC via impact weighting	6
6	Channel Estimators for VOC Components	6
6.1	One-loop channel (FOCUS)	6
6.1.1	Flip probability under Gaussian beliefs	6
6.1.2	Finite-temperature one-loop bonus	7
6.2	Nested sampling channel (EXPAND)	7
6.2.1	Hidden-mode probability	7
6.2.2	NS gate (when NS is pointless)	7
6.3	Discrete curvature channel (MERGE / GEOM)	8
6.3.1	Holonomy residual curvature on the transposition graph	8
6.3.2	Intrinsic tree curvature via graph Ricci curvature	8
6.4	Free-energy/surprise channel (FEP modulator)	8
6.5	Explicit QUARTZ VOC instantiation	8
6.5.1	Network outputs and calibrated scale	8
6.5.2	FOCUS: flip probability and gain	9
6.5.3	EXPAND: hidden-mode probability and improvement	9
6.5.4	MERGE: curvature and reconciliation gain	10
6.5.5	Global scheduling	10
6.5.6	Deterministic halting (PI-style time targeting)	10
7	Halting and Termination Guarantees	10
7.1	Deterministic halting controller	10
7.2	Calibration-based error control	11
8	Non-Gaussian Rollout Noise and Higher-Order Corrections	11
8.1	Diagnostic heavy-tail score	11
8.2	Higher-order cumulant correction	11
9	Algorithms (Pseudocode)	12
9.1	Baseline: Neural MCTS with TT, PW, and tree parallelization	12
9.2	QUARTZ: GVOC-driven compute allocation	12
9.3	Task execution details	12
10	SearchController API Specification (Rigorous)	12
10.1	Purpose and scope	12
10.2	Abstract interface	14
10.3	Formal semantics and invariants	15
10.3.1	Session state machine	15
10.3.2	Thread-safety contract	16
10.3.3	Idempotence and duplicate work	16
10.3.4	Canonical path requirement	16

10.3.5	Deterministic halting	16
10.4	Controller-specific details: Baseline vs QUARTZ	16
10.5	Complexity and performance contracts	17
10.6	Data-model contracts for correctness and speed	17
10.6.1	TreeStore: node/edge state and versioning	17
10.6.2	TranspositionTable (TT) contract	18
10.6.3	Virtual loss contract	18
10.7	SearchResult and logging metrics	18
10.7.1	SearchResult contract	18
10.7.2	Mandatory log/KPI set	19
10.8	Reference implementation strategy	19
11	Implementation Notes (CPU-centric)	19
12	Limitations and Open Issues	20
13	Conclusion	20

1 Problem Setting and Design Constraints

1.1 Game model

We consider deterministic, two-player, zero-sum, perfect-information board games. Let $s \in \mathcal{S}$ denote a state, $A(s)$ the finite legal action set, and $T : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ the deterministic transition. A terminal state $s \in \mathcal{S}_{\text{term}}$ yields utility $u(s) \in \{-1, 0, +1\}$ from the perspective of the player to move at s (draw allowed).

1.2 Neural prior and value

A neural network with parameters θ provides

$$\pi_{\theta}(a \mid s) \in [0, 1], \quad \sum_{a \in A(s)} \pi_{\theta}(a \mid s) = 1, \quad v_{\theta}(s) \in [-1, 1],$$

computed in a single forward pass (*policy+value once*).

1.3 Compute constraints

Training may use high-performance GPU servers; inference must run on consumer CPUs (AVX2 assumed). A move-level time budget has a *target* $T_{\text{target}} \approx 5\text{s}$ and a *hard cap* $T_{\text{max}} \approx 30\text{s}$, with a desired distribution: easy moves $< 1\text{s}$, hard moves up to T_{max} , while maintaining the target average via adaptive halting.

1.4 Baseline fairness contract

To fairly measure algorithmic contributions beyond standard MCTS+NN, any baseline must include: (i) AlphaZero-style PUCT selection, (ii) progressive widening (or equivalent branch control) for large action sets, (iii) tree-parallel MCTS with virtual loss, (iv) a shared transposition table (TT), (v) tree reuse across moves, and (vi) identical neural architectures and inference backends. Only the compute-allocation logic (QUARTZ) may differ.

2 Preliminaries: MCTS, PUCT, and Parallelism

2.1 Edge statistics

Each node s maintains per-action statistics:

$$N(s, a) \in \mathbb{N}, \quad W(s, a) \in \mathbb{R}, \quad Q(s, a) = \frac{W(s, a)}{\max\{1, N(s, a)\}}.$$

Let $N(s) = \sum_a N(s, a)$.

2.2 PUCT selection

Standard AlphaZero-style selection uses

$$\text{PUCT}(s, a) = Q(s, a) + c_{\text{puct}} \pi_{\theta}(a \mid s) \frac{\sqrt{N(s)}}{1 + N(s, a)}.$$

Tree parallelization runs many simulations concurrently, typically with *virtual loss* to reduce redundant work; see [11] for a classic discussion.

2.3 Progressive widening

When $|A(s)|$ is large, progressive widening controls branching by limiting the number of expanded children:

$$|\text{children}(s)| \leq m(s) := \min\{|A(s)|, k_0 + k_1 N(s)^\alpha\},$$

opening new children in order of decreasing $\pi_{\theta}(a \mid s)$ (or other fixed baseline rule). This is a standard technique for large/continuous action spaces [4, 5].

3 Unified Latent-Utility Belief Model

3.1 Latent utility and observations

For each state-action pair (s, a) we posit a latent utility $U(s, a) \in \mathbb{R}$ representing the (unknown) root-relevant advantage of choosing a at s . The neural network provides a *prior* for $U(s, a)$ through a location-scale family:

$$U(s, a) \sim \mathcal{P}(\mu_{\theta}(s, a), \sigma_{\theta}(s, a)),$$

where μ_{θ} and (optionally) σ_{θ} are outputs or derived uncertainty proxies. Search generates noisy observations $\tilde{U}$ via leaf evaluation and backup:

$$\tilde{U}_i(s, a) = U(s, a) + \varepsilon_i, \quad \varepsilon_i \sim \text{Noise}(s, a).$$

Unlike many treatments, $\text{Noise}(s, a)$ is allowed to be heavy-tailed or multimodal (e.g., due to tactical traps).

3.2 Posterior sketch and robust moments

We do not require an exact parametric posterior. Instead, each edge maintains robust estimates of location and scale from observed returns. In the Gaussian special case, Welford updates yield unbiased estimates of $(\hat{\mu}, \hat{\sigma}^2)$. For heavy tails, we recommend robust alternatives (median-of-means, trimmed means, or Student- t scale), but QUARTZ uses them only to gate nested sampling and correct one-loop terms (Section 8).

4 Thermodynamic View: Finite Temperature and Free Energy

4.1 Variational free-energy form

Let $\mu_a := \mathbb{E}[U(s, a) \mid \text{data}]$ be the current posterior mean at a fixed state s . Let $\pi(a) \equiv \pi_\theta(a \mid s)$ denote the neural prior. For any inverse temperature $\beta > 0$, define the functional

$$\mathcal{L}_\beta(q) := \sum_{a \in A(s)} q(a) \mu_a - \frac{1}{\beta} \text{KL}(q \parallel \pi), \quad q \in \Delta(A(s)), \quad (1)$$

where $\Delta(A(s))$ is the probability simplex.

Proposition 1 (Finite-temperature optimum and log-partition). *The maximizer of (1) is*

$$q_\beta(a) = \frac{\pi(a) \exp(\beta \mu_a)}{Z(\beta)}, \quad Z(\beta) := \sum_a \pi(a) \exp(\beta \mu_a),$$

and the optimal value is $\max_q \mathcal{L}_\beta(q) = \frac{1}{\beta} \log Z(\beta)$.

Proof. Introduce a Lagrange multiplier λ for $\sum_a q(a) = 1$ and set derivatives to zero:

$$\frac{\partial}{\partial q(a)} \left(\sum_{a'} q(a') \mu_{a'} - \frac{1}{\beta} \sum_{a'} q(a') \log \frac{q(a')}{\pi(a')} + \lambda (\sum_{a'} q(a') - 1) \right) = 0.$$

This yields $\mu_a - \frac{1}{\beta} (\log q(a) - \log \pi(a) + 1) + \lambda = 0$, hence $q(a) \propto \pi(a) \exp(\beta \mu_a)$. Substituting back gives $\frac{1}{\beta} \log Z(\beta)$. $\square$

Interpretation. Equation (1) matches the standard variational free-energy / evidence lower bound form used in the free-energy principle literature [7], with π acting as a prior over actions and μ as (negative) energy.

4.2 One-loop correction as a cumulant term

Let U_a be random under the posterior. The thermodynamic objective depends on the log-partition $\log \sum_a \pi(a) e^{\beta U_a}$. A Gaussian (Laplace) approximation yields the leading term $\log Z(\beta)$ plus a fluctuation correction analogous to the one-loop effective action (a log-determinant/Hessian term). In QUARTZ we use this idea not by computing Hessians explicitly, but by injecting a *one-loop bonus* derived from low-order cumulants (Section 6.1).

5 Rational Metareasoning and Value of Computation

5.1 Compute actions

At any moment the search process may choose a compute action c at a node s :

Action c	Meaning
FOCUS	spend compute to reduce uncertainty among already-expanded actions at s
EXPAND	evaluate or add a new action $a \notin \text{live}(s)$ (widening)
MERGE	reconcile inconsistent estimates for s via transpositions / curvature
STOP	allocate no further compute to s (or halt the whole search)

Each action has an observed (measured) cost $\text{cost}(c)$ in wall-clock time (tracked by EMA).

5.2 Value of computation (VOC)

Let s_0 denote the root. Let a^* be the action currently believed best at the root, and a_c^* the action that would be selected after performing compute action c and updating beliefs. The *root-level* value of computation is

$$\text{VOC}(c) := \mathbb{E}[\mu_{s_0}(a_c^*) - \mu_{s_0}(a^*)] - \text{cost}(c). \quad (2)$$

This is a myopic form of rational metareasoning (Russell–Wefald VOC) [1].

5.3 Global VOC via impact weighting

Compute actions are executed at internal nodes $s \neq s_0$ but matter insofar as they change the root decision. QUARTZ uses an impact weight

$$w_{\text{imp}}(s) := \frac{N(s)}{N(s_0)} \in [0, 1],$$

interpretable as an empirical probability that a simulation from the root passes through s under the current search policy. The *Global VOC* is

$$\text{GVOC}(c, s) := w_{\text{imp}}(s) \text{VOC}(c \mid s), \quad (3)$$

where $\text{VOC}(c \mid s)$ is a local proxy for root improvement. The scheduler repeatedly chooses $(c^*, s^*) = \arg \max_{c, s} \text{GVOC}(c, s)$ and stops when the maximum is non-positive.

6 Channel Estimators for VOC Components

QUARTZ decomposes

$$\text{VOC}(c \mid s) \approx P_{\text{change}}(c, s) \cdot \mathbb{E}[\Delta(c, s) \mid \text{change}] - \text{cost}(c),$$

where P_{change} is the probability that the root-relevant decision changes as a consequence of compute action c at node s , and Δ is the improvement conditional on change.

6.1 One-loop channel (FOCUS)

6.1.1 Flip probability under Gaussian beliefs

Let a_1, a_2 be the current top-2 actions at node s under posterior means, with posterior moments (μ_i, σ_i^2) for $U_i := U(s, a_i)$. Define $\Delta := U_1 - U_2$ with mean $\mu_\Delta = \mu_1 - \mu_2$ and variance

$$\sigma_\Delta^2 = \sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2,$$

where ρ is a correlation coefficient. A conservative upper bound sets $\rho = 0$ when correlation is unknown.

Lemma 1 (Gaussian flip probability). *If $\Delta \sim \mathcal{N}(\mu_\Delta, \sigma_\Delta^2)$ then*

$$P_{\text{flip}}(s) := \Pr(\Delta < 0) = \Phi\left(-\frac{\mu_\Delta}{\sigma_\Delta}\right),$$

where Φ is the standard normal CDF.

Proof. Standardize: $\Pr(\Delta < 0) = \Pr((\Delta - \mu_\Delta)/\sigma_\Delta < -\mu_\Delta/\sigma_\Delta) = \Phi(-\mu_\Delta/\sigma_\Delta)$. $\square$

6.1.2 Finite-temperature one-loop bonus

Selection can be viewed as maximizing $\mathcal{L}_\beta(q)$ (Proposition 1) rather than a hard argmax. QUARTZ injects a correction term into selection:

$$\text{score}(s, a) = \text{PUCT}(s, a) + B_{1\text{loop}}(s, a),$$

where $B_{1\text{loop}}$ is derived from low-order cumulants of return noise. In the simplest robust form,

$$B_{1\text{loop}}(s, a) := \lambda_{1\text{loop}} \cdot \log(1 + \hat{\sigma}(s, a)),$$

with $\hat{\sigma}$ a robust scale estimate and $\lambda_{1\text{loop}}$ auto-normalized by a running median to avoid introducing a free hyperparameter.

6.2 Nested sampling channel (EXPAND)

6.2.1 Hidden-mode probability

Let $\mathcal{L}_K(s) \subseteq A(s)$ be the current *live set* (expanded set) of size K . Define the outside prior mass

$$m_{\text{out}}(s) := 1 - \sum_{a \in \mathcal{L}_K(s)} \pi(a).$$

Let $U_{\text{out}}^* := \max_{a \notin \mathcal{L}_K(s)} U(s, a)$ and $U_1 := \max_{a \in \mathcal{L}_K(s)} U(s, a)$. The ideal quantity is

$$P_{\text{hidden}}(s) := \Pr(U_{\text{out}}^* > U_1).$$

Exact computation is hard for large discrete action sets, so we use an NS-inspired Poisson approximation.

Lemma 2 (Poisson approximation for hidden modes). *Assume that outside actions produce improvements as rare events with tail probability $p_{\text{tail}}(s)$ per unit outside prior mass, and that effective outside “trials” scale with $m_{\text{out}}(s)$. Then*

$$P_{\text{hidden}}(s) \approx 1 - \exp(-m_{\text{out}}(s) p_{\text{tail}}(s)).$$

Proof sketch. Model the number of outside improving actions encountered as a Poisson random variable with rate $\lambda = m_{\text{out}} p_{\text{tail}}$. Then $P(\text{at least one}) = 1 - e^{-\lambda}$. $\square$

6.2.2 NS gate (when NS is pointless)

If rollout noise is light-tailed and the policy prior is effectively unimodal, then P_{hidden} is negligible and explicit NS-style probing wastes compute. QUARTZ uses a gate

$$\text{NS_gate}(s) = \mathbf{1}\{\tilde{H}(s) \vee \tilde{S}(s) \vee \tilde{T}(s)\},$$

where $\tilde{H}$ is a normalized multimodality proxy (e.g., entropy/peak ratio), $\tilde{S}$ is normalized prior-posterior surprise (Section 6.4), and $\tilde{T}$ is a normalized heavy-tail proxy (Section 8). Normalization by running medians removes explicit thresholds, leaving only time budgets and a live-set cap.

6.3 Discrete curvature channel (MERGE / GEOM)

6.3.1 Holonomy residual curvature on the transposition graph

If state s is reachable by multiple paths (TT hit count ≥ 2), store incoming backed-up values $\{Q_{\gamma_i}(s)\}_{i=1}^m$ associated with distinct paths γ_i . Define

$$R_{\text{TT}}(s) := \text{Var}(Q_{\gamma_1}(s), \dots, Q_{\gamma_m}(s)).$$

This mirrors the idea that curvature is detected by non-trivial holonomy around loops.

6.3.2 Intrinsic tree curvature via graph Ricci curvature

In games where TT is sparse (e.g., Go 19x19), we instead define curvature intrinsically on the local tree-graph using coarse Ricci curvature (Ollivier–Ricci) [12] or Forman-style curvature on edges [13]. Negative curvature indicates local expansion / information divergence, suggesting “hard” regions where additional compute is valuable.

6.4 Free-energy/surprise channel (FEP modulator)

Define the empirical tree policy (posterior over actions) at node s from visit counts:

$$q_T(a \mid s) := \frac{N(s, a) + \varepsilon}{N(s) + |A(s)|\varepsilon},$$

with a tiny smoothing constant ε (implementation constant, not tuned). Define a smoothed prior $\pi_\varepsilon = (1 - \varepsilon)\pi + \varepsilon/|A(s)|$. The surprise (free-energy proxy) is

$$S(s) := \text{KL}(q_T(\cdot \mid s) \parallel \pi_\varepsilon(\cdot \mid s)).$$

QUARTZ uses $S(s)$ to (i) modulate the expected gain Δ (model mismatch implies potential improvement), and (ii) gate nested sampling (large surprise suggests a missed mode).

6.5 Explicit QUARTZ VOC instantiation

This subsection states the concrete scalar definitions. The goal is to (i) pin down every quantity precisely, (ii) minimize free hyperparameters by using running statistics (EMA/medians), and (iii) make the controller implementable without scanning all actions.

6.5.1 Network outputs and calibrated scale

For each expanded edge (s, a) we maintain a posterior mean $\mu(s, a)$ and a scale $\sigma_{\text{net}}(s, a)$ (either predicted by the network or derived from rollout variance). To correct systematic miscalibration, maintain a depth-bucketed scalar κ_b with buckets $b \in \{\text{SHALLOW}, \text{MID}, \text{DEEP}\}$. Define

$$\sigma(s, a) := \kappa_{b(s)} \sigma_{\text{net}}(s, a).$$

The buckets are defined by depth ranges and do not introduce tuned thresholds; they are part of the implementation contract.

6.5.2 FOCUS: flip probability and gain

Let (a_1, a_2) be the current top-2 actions at node s by mean utility. Define

$$\mu_\Delta(s) := \mu(s, a_1) - \mu(s, a_2), \quad \sigma_\Delta^2(s) := \sigma^2(s, a_1) + \sigma^2(s, a_2),$$

using the conservative $\rho = 0$ bound. Then

$$P_{\text{flip}}(s) := \Phi\left(-\frac{\mu_\Delta(s)}{\sigma_\Delta(s)}\right).$$

Define a free-energy modulated gain

$$\mathbb{E}[\Delta_{\text{focus}}(s)] := \sigma_\Delta(s) \left(1 + \frac{S(s)}{S_0}\right),$$

where S_0 is the running median of surprise values (Section 6.4). The local VOC is

$$\text{VOC}_{\text{FOCUS}}(s) := P_{\text{flip}}(s) \mathbb{E}[\Delta_{\text{focus}}(s)] - \text{cost}_{\text{FOCUS}}. \quad (4)$$

6.5.3 EXPAND: hidden-mode probability and improvement

Let $\mathcal{L}_K(s)$ be the current live set (expanded children). Define outside mass

$$m_{\text{out}}(s) := 1 - \sum_{a \in \mathcal{L}_K(s)} \pi(a \mid s).$$

Let a_{out}^* be a cheap proposal for the best outside action (e.g., top- K by π not already in $\mathcal{L}_K$). Define a tail event probability

$$p_{\text{tail}}(s) := \Phi\left(\frac{\mu(s, a_{\text{out}}^*) - \mu(s, a_1)}{\sqrt{\sigma^2(s, a_{\text{out}}^*) + \sigma^2(s, a_1)}}\right).$$

Then

$$P_{\text{hidden}}(s) := 1 - \exp(-m_{\text{out}}(s) p_{\text{tail}}(s)).$$

A Gaussian truncated-improvement approximation gives

$$\mathbb{E}[\Delta_{\text{expand}}(s)] := \sigma_{\text{comb}} [\phi(z) + z\Phi(z)], \quad z = \frac{\mu^* - \mu_1}{\sigma_{\text{comb}}}, \quad \sigma_{\text{comb}} = \sqrt{\sigma^{*2} + \sigma_1^2},$$

where (μ^*, σ^*) correspond to a_{out}^* , and ϕ is the standard normal PDF. The local VOC is

$$\text{VOC}_{\text{EXPAND}}(s) := P_{\text{hidden}}(s) \mathbb{E}[\Delta_{\text{expand}}(s)] - \text{cost}_{\text{EXPAND}}. \quad (5)$$

Hyperparameter-free filtering when NS is irrelevant. Notice that (5) itself filters NS: if m_{out} is tiny or p_{tail} is tiny, then P_{hidden} is tiny and the VOC becomes negative. For efficiency one can additionally use the upper bound

$$\text{VOC}_{\text{EXPAND}}(s) \leq m_{\text{out}}(s) p_{\text{tail}}(s) \mathbb{E}[\Delta_{\text{expand}}(s)] - \text{cost}_{\text{EXPAND}},$$

and skip exact computation whenever this bound is non-positive.

6.5.4 MERGE: curvature and reconciliation gain

When TT hit count is at least 2, maintain a curvature proxy $R(s)$, either as the incoming-path variance $R_{\text{TT}}(s)$ or the cycle-residual variance variant. Define

$$P_{\text{merge}}(s) := \min \left\{ 1, \frac{R(s)}{R_0} \right\}, \quad \mathbb{E}[\Delta_{\text{merge}}(s)] := \sqrt{R(s)},$$

with R_0 a running median of observed curvatures. The local VOC is

$$\text{VOC}_{\text{MERGE}}(s) := P_{\text{merge}}(s) \mathbb{E}[\Delta_{\text{merge}}(s)] - \text{cost}_{\text{MERGE}}. \quad (6)$$

6.5.5 Global scheduling

For $c \in \{\text{FOCUS}, \text{EXPAND}, \text{MERGE}\}$ define

$$\text{GVOC}(c, s) := \frac{N(s)}{N(s_0)} \text{VOC}_c(s),$$

and choose $(c^*, s^*) = \arg \max_{c,s} \text{GVOC}(c, s)$. Stop allocating compute when the maximum is non-positive or when the time cap is reached.

6.5.6 Deterministic halting (PI-style time targeting)

Define

$$\text{Conf}(t) := (1 - P_{\text{flip}}(s_0)) (1 - P_{\text{hidden}}(s_0)) \max\{0, 1 - S(s_0)/S_0\}.$$

QUARTZ halts if $\text{Conf}(t) \geq \theta_{\text{conf}}$ or elapsed time exceeds T_{max} or $\max \text{GVOC} \leq 0$. The threshold is adapted online to hit the target average time:

$$\theta_{\text{conf}} \leftarrow \theta_{\text{conf}} + \eta (T_{\text{actual}} - T_{\text{target}}),$$

where η is a small fixed step size (implementation constant).

7 Halting and Termination Guarantees

7.1 Deterministic halting controller

Define a confidence proxy at the root:

$$\text{Conf}(t) := (1 - P_{\text{flip}}(s_0)) (1 - P_{\text{hidden}}(s_0)) \max\{0, 1 - S(s_0)/S_0\},$$

where S_0 is a running median of surprise values (auto-scaled). QUARTZ halts if any of: (i) elapsed time exceeds T_{max} , (ii) $\max_{c,s} \text{GVOC}(c, s) \leq 0$, or (iii) $\text{Conf}(t)$ exceeds an adaptive threshold tuned to meet T_{target} on average.

Theorem 1 (Eventual termination under diminishing returns). *Assume: (A1) For every node-action, each additional FOCUS reduces posterior scale in expectation; (A2) EXPAND eventually exhausts useful actions so that $m_{\text{out}}(s) \rightarrow 0$; (A3) MERGE reduces disagreement so $R(s) \rightarrow 0$; (A4) costs $\text{cost}(c)$ are bounded below by a positive constant. Then, for any fixed root, $\max_{c,s} \text{GVOC}(c, s)$ becomes non-positive after finitely many compute steps, hence QUARTZ halts (or earlier due to time caps).*

Proof. Under (A1)–(A3), the benefit terms $P_{\text{change}} \cdot \mathbb{E}[\Delta]$ in every local VOC proxy converge to 0 as uncertainties vanish, outside mass vanishes, and disagreement vanishes. By (A4), subtracting positive cost yields negative VOC for sufficiently large compute. Multiplying by $w_{\text{imp}}(s) \in [0, 1]$ preserves non-positivity, so the global maximum becomes non-positive. $\square$

7.2 Calibration-based error control

Let $\hat{P}_{\text{flip}}$ be the predicted flip probability. In practice it must be calibrated (e.g., via temperature scaling or online scalars). Expected calibration error (ECE) is a common scalar measure [8].

Proposition 2 (Halting error bound under calibration). *Suppose a halting rule stops when $\hat{P}_{\text{flip}}(s_0) \leq \delta$ for some $\delta \in (0, 1)$. Assume $\hat{P}_{\text{flip}}$ is ϵ -calibrated in the sense that for events of the form $\{\hat{P}_{\text{flip}} \in B\}$, the difference between predicted and empirical probabilities is at most ϵ on average ($ECE \leq \epsilon$). Then the probability that the true best action at the root differs from the current best at halting is at most $\delta + \epsilon$ (up to binning approximation).*

Proof sketch. At stopping, $\hat{P}_{\text{flip}} \leq \delta$. Calibration implies $\Pr(\text{flip} \mid \hat{P}_{\text{flip}} \leq \delta) \leq \delta + \epsilon$ in expectation over calibration bins, yielding the bound. $\square$

8 Non-Gaussian Rollout Noise and Higher-Order Corrections

QUARTZ explicitly anticipates regimes where Gaussian assumptions fail:

- **Heavy tails:** tactical traps create rare but decisive value swings.
- **Multimodality:** policy priors may have separated peaks (multiple plausible plans).

The guiding principle is: *use Gaussian one-loop approximations by default, but switch to NS probing and higher-order cumulant corrections only when diagnostic statistics indicate failure.*

8.1 Diagnostic heavy-tail score

Let $\{z_i\}$ be backed-up returns for an edge. Define a robust scale MAD and a classical scale $\hat{\sigma}$. A simple heavy-tail proxy is

$$T(s, a) := \frac{\hat{\sigma}(s, a)}{\text{MAD}(s, a) + 10^{-8}}.$$

Large T suggests outliers/heavy tails. QUARTZ normalizes T by a running median across the search to avoid fixed thresholds.

8.2 Higher-order cumulant correction

Let $K_{\Delta}(t) = \log \mathbb{E}[e^{t\Delta}]$ be the cumulant generating function of $\Delta = U_1 - U_2$. The Gaussian approximation uses only the first two cumulants. When heavy tails are detected, QUARTZ allows an optional saddlepoint correction to P_{flip} based on K_{Δ} (using empirical cumulants up to order 3 or 4 when stable). This preserves a “generalized path integral” viewpoint without committing to expensive symbolic QFT operations.

Algorithm 1 Baseline PUCT MCTS (tree-parallel, shared TT, progressive widening)

```
1: procedure SEARCH( $s_0, T_{\text{budget}}, P$ )
2:    $root \leftarrow TT.\text{getOrCreate}(s_0)$ 
3:   INITROOT( $root$ ) ▷ cache priors via NN once if needed
4:    $deadline \leftarrow now() + T_{\text{budget}}$ 
5:   for all  $p \in \{1, \dots, P\}$  in parallel do
6:     while  $now() < deadline$  do
7:       SIMULATE( $root$ )
8:     end while
9:   end for
10:  return  $\pi(a) \propto N(root, a)$ 
11: end procedure
12: procedure SIMULATE( $v$ )
13:   $path \leftarrow []$ 
14:  while  $v.\text{expanded}$  and not  $\text{terminal}(v)$  do
15:     $a \leftarrow \arg \max_{a \in v.\text{children}} \text{PUCT}(v, a)$ 
16:    APPLYVIRTUALLOSS( $v, a$ )
17:     $path.\text{append}((v, a))$ 
18:     $v \leftarrow TT.\text{getOrCreate}(T(v, a))$ 
19:  end while
20:  if not  $\text{terminal}(v)$  then
21:    EXPANDPW( $v$ ) ▷ open next child by prior if allowed
22:     $(P, vval) \leftarrow NN(v.\text{state})$ 
23:     $v.\text{setPriors}(P)$ 
24:  else
25:     $vval \leftarrow u(v.\text{state})$ 
26:  end if
27:  for  $(u, a)$  in  $\text{reverse}(path)$  do
28:    REMOVEVIRTUALLOSS( $u, a$ )
29:    BACKUP( $u, a, vval$ )
30:     $vval \leftarrow -vval$ 
31:  end for
32: end procedure
```

9 Algorithms (Pseudocode)

9.1 Baseline: Neural MCTS with TT, PW, and tree parallelization

9.2 QUARTZ: GVOC-driven compute allocation

9.3 Task execution details

10 SearchController API Specification (Rigorous)

10.1 Purpose and scope

The **SearchController** is a pure *compute-allocation controller*. It does not define game rules, does not own the neural network, and does not hard-code MCTS details. Instead it orchestrates *what*

Algorithm 2 QUARTZ search loop (GVOC scheduler with compute actions)

```
1: procedure QUARTZSEARCH( $s_0, T_{\text{target}}, T_{\text{max}}, P$ )
2:    $ctx \leftarrow \text{INITCONTEXT}(s_0)$ 
3:    $deadline \leftarrow \text{now}() + T_{\text{max}}$ 
4:   SCHEDULERINIT( $ctx$ ) ▷ seed heap with initial candidates
5:   for all  $p \in \{1, \dots, P\}$  in parallel do
6:     while  $\text{now}() < deadline$  do
7:        $task \leftarrow ctx.scheduler.popBest()$ 
8:       if  $task = \emptyset$  then
9:         break
10:      end if
11:      if not VALIDATE( $ctx, task$ ) then
12:        continue
13:      end if
14:       $result \leftarrow \text{EXECUTETASK}(ctx, task)$ 
15:      COMMIT( $ctx, task, result$ )
16:      RESCHEDULENEIGHBORHOOD( $ctx, task$ )
17:      if SHOULDHALT( $ctx$ ) then
18:        break
19:      end if
20:    end while
21:  end for
22:  return FINALIZE( $ctx$ )
23: end procedure
```

compute to do next, under a strict time budget, in a multithreaded setting.

Key responsibilities.

1. Maintain an internal scheduling state (heap/queue) of pending compute tasks.
2. Read-only access to the shared **TreeStore** to compute GVOC features.
3. Provide thread-safe task acquisition for worker threads.
4. Accept task completion events and update scheduling state.
5. Implement deterministic halting under time caps and controller-specific criteria.

Non-responsibilities.

1. It does not perform neural inference; that is delegated to **EvalBackend**.
2. It does not apply game transitions; that is delegated to **Game**.
3. It does not own TT memory; that is delegated to **TranspositionTable**.
4. It does not decide training; it operates inside self-play or inference episodes.

Algorithm 3 ExecuteTask (compute actions)

```
1: function EXECUTETASK(ctx, task)
2:   (c, s)  $\leftarrow$  (task.kind, task.target)
3:   if c = FOCUS then
4:     path  $\leftarrow$  FOLLOWCANONICALPATH(ctx.tree, s)
5:     return FOCUSEDSIMULATIONFROM(ctx, path, s)
6:   else if c = EXPAND then
7:     if not NSGATE(ctx, s) then
8:       return BASELINEPWEXPAND(ctx, s)
9:     end if
10:    a  $\leftarrow$  PROPOSEOUTSIDEACTION(ctx, s)
11:    return EXPANDANDEVAL(ctx, s, a)
12:   else if c = MERGE then
13:     return MERGETRANSPOSITIONS(ctx, s)
14:   else if c = GEOM then
15:     return UPDATECURVATURE(ctx, s)
16:   end if
17: end function
```

10.2 Abstract interface

We specify a language-agnostic API with pre/postconditions. All methods are assumed to be callable from C++ with Python bindings if desired.

Listing 1: SearchController interface (spec)

```
interface SearchController {
  // Create a new per-move session.
  SearchSession begin_search(GameState root,
                             SearchLimits limits,
                             SearchSeed seed);

  // Controller identity for logging/reproducibility.
  string name();
  Version version();
}

struct SearchLimits {
  duration target_time; // e.g., 5s
  duration hard_cap_time; // e.g., 30s
  int32 thread_count; // P
  int32 max_nodes; // optional safety
  int32 max_memory_mb; // optional safety
}

interface SearchSession {
  // Thread-safe: can be called concurrently by worker threads.
  // Returns NONE if (i) should halt, or (ii) no useful task exists.
  optional<SearchTask> acquire_task(TimePoint now);

  // Thread-safe. The engine calls this after executing the task.
  // Must be idempotent: duplicate completions are ignored safely.
```

```

void complete_task(SearchTask task, TaskResult result);

// Thread-safe. Deterministic: given same internal state and now,
// returns the same decision independent of thread interleavings.
bool should_halt(TimePoint now);

// Not thread-safe; must be called once after workers stop.
SearchResult finalize();
}

enum TaskKind { FOCUS, EXPAND, MERGE, GEOM, NOOP }

struct SearchTask {
    TaskId id;
    TaskKind kind;
    NodeId target;

    // For FOCUS/EXPAND tasks: a canonical path from root to target.
    // This makes the task's effect on root statistics well-defined.
    PathHint spine;

    // For EXPAND: the action to be evaluated/added (may be unset if
    // controller requests the engine to choose via a callback).
    optional<ActionId> action;

    // Version stamp for lazy invalidation: if node.version differs,
    // the task is stale and must be discarded.
    uint64 node_version;

    // Priority (e.g., GVOC). Only for logging; not used for validity.
    float priority;
}

struct TaskResult {
    // Tagged union.
    TaskKind kind;

    // FOCUS: simulation path, leaf value, and time cost.
    optional<SimulationTrace> sim;
    // EXPAND: added child + NN evaluation outputs.
    optional<ExpandTrace> exp;
    // MERGE/GEOM: updated curvature stats.
    optional<MergeTrace> merge;

    duration wall_time;
}

```

10.3 Formal semantics and invariants

10.3.1 Session state machine

A `SearchSession` is a state machine with states:

$$\text{INIT} \rightarrow \text{RUNNING} \rightarrow \text{HALTING} \rightarrow \text{FINALIZED}.$$

- INIT: root is created, priors possibly requested, scheduler seeded.
- RUNNING: tasks are issued and completed concurrently.
- HALTING: `should_halt` becomes true; `acquire_task` returns none.
- FINALIZED: `finalize` returns an action distribution and closes the session.

10.3.2 Thread-safety contract

Definition 1 (Thread-safe method). *A method is thread-safe if concurrent calls are well-defined and produce a state equivalent to some serial interleaving of those calls (linearizability).*

The following methods must be thread-safe and linearizable: `acquire_task`, `complete_task`, `should_halt`. The method `finalize` is *not* thread-safe and must be called only after workers join.

10.3.3 Idempotence and duplicate work

Because tasks may be duplicated (e.g., stale heap entries), `complete_task` must be idempotent:

$$\text{complete_task}(\text{task}, \text{result}) \text{ applied twice} \equiv \text{applied once.}$$

Implementation must use a task-id registry or version checks to ignore duplicates safely.

10.3.4 Canonical path requirement

For non-root tasks, QUARTZ requires a *canonical spine path* from root to the target node. This ensures that a FOCUS or EXPAND at node s has a well-defined effect on root statistics by executing the simulation starting at the root and forcing traversal along the spine before normal selection. Without this requirement, “local” simulations at s would not necessarily change root counts, breaking the interpretation of $w_{\text{imp}}(s)$ and GVOC.

10.3.5 Deterministic halting

To meet strict move-time control, `should_halt` must be deterministic given session state and current time. In QUARTZ this is achieved by: (i) a hard wall-clock cap, (ii) a non-positive maximum GVOC condition, and (iii) a deterministic confidence threshold controller. Randomized halting (Bernoulli sampling) is explicitly disallowed in the consumer-facing engine.

10.4 Controller-specific details: Baseline vs QUARTZ

A baseline PUCT controller is obtained by implementing the same API but restricting tasks to:

$$\text{TaskKind} = \text{FOCUS}, \quad \text{target} = \text{root}, \quad \text{priority} = 0,$$

with `should_halt` depending only on elapsed time. This makes baseline and QUARTZ comparable under identical infrastructure.

10.5 Complexity and performance contracts

- `acquire_task` should be $O(\log M)$ where M is heap size; constant factors must be bounded (no scanning all nodes).
- `complete_task` should be amortized $O(\log M)$ plus localized rescheduling.
- The controller must never require an $O(|A(s)|)$ full scan on the critical path; all action selection must be top- K or sampled.

10.6 Data-model contracts for correctness and speed

The `SearchController` API is only meaningful if the underlying data model is specified precisely. This subsection fixes the required invariants for `TreeStore`, `TranspositionTable`, and the measurement/caching that QUARTZ relies on.

10.6.1 TreeStore: node/edge state and versioning

Node identity. Each unique game state s in the current search tree is assigned a stable `NodeId`. A `NodeId` never changes meaning within a session.

Versioning (lazy invalidation). Each node maintains an atomic `version` counter. Any mutation that may change GVOC-relevant quantities (visit counts, top-2 actions, surprise, curvature, live-set membership) increments `version`. A `SearchTask` with `node_version` is valid iff it matches the node’s current `version` at acquisition time.

Required fields. The controller assumes the following minimal per-node state:

Listing 2: TreeStore node state (required)

```
struct Node {
    NodeId id;
    GameState state;

    // Visit totals
    atomic<int32> N;

    // Action index set for live children (size K <= K_max).
    ActionId live_actions[K_max];

    // Per-live-action edge stats (SoA strongly preferred).
    atomic<int32> N_a[K_max];
    atomic<float> W_a[K_max];
    atomic<float> Q_a[K_max];

    // Optional: online variance / robust scale.
    atomic<float> M2_a[K_max];
    atomic<float> sigma_a[K_max];

    // Cached top-2 indices for O(1) flip updates.
    atomic<int16> top1_idx;
    atomic<int16> top2_idx;
```

```

// FEP caches for O(1) KL updates (implementation-defined).
atomic<float> surprise_S;

// TT / geometry caches.
atomic<int16> tt_hits;
atomic<float> curvature_R;

// Canonical parent pointer for spine path.
atomic<NodeId> parent;

// Version for lazy invalidation.
atomic<uint64> version;
}

```

Atomicity requirement. At minimum, updates to (N, N_a, W_a, Q_a) must be linearizable at the edge level, so that concurrent selection reads see a consistent state. This can be achieved with: (i) per-node spinlocks, or (ii) atomic fetch-add on counts and atomic add on sums, followed by relaxed recomputation of derived caches (top-2, S , R) under the same lock.

10.6.2 TranspositionTable (TT) contract

The TT maps a game-state hash to a canonical `NodeId`. It must support concurrent `getOrCreate` calls. The TT must also expose *alias edges* (alternative parents) needed by MERGE tasks.

10.6.3 Virtual loss contract

If the engine uses virtual loss, it must satisfy:

- **Acquire:** before a worker traverses an edge (s, a) , it atomically increments a virtual-visit counter $N^{\text{vl}}(s, a)$.
- **Release:** after backup, it decrements $N^{\text{vl}}(s, a)$.
- **Selection:** the selection formula reads $N(s, a) + N^{\text{vl}}(s, a)$.

This ensures that parallel workers avoid redundant leaf selection while preserving unbiased backups [11].

10.7 SearchResult and logging metrics

10.7.1 SearchResult contract

The output of `finalize()` must include enough information for training (self-play) and for reproducible evaluation.

Listing 3: SearchResult (required fields)

```

struct SearchResult {
    ActionId chosen_action;

    // Root policy for training (e.g., proportional to visits).
    vector<pair<ActionId, float>> root_policy;
}

```

```

// Root value estimate from the player's perspective.
float root_value;

// Timing
duration elapsed;
int32 threads_used;

// Diagnostics (for ablation and debugging)
float p_flip_root;
float p_hidden_root;
float surprise_root;
float max_gvoc;
float ece_estimate;

// Throughput
float sims_per_sec;
float evals_per_sec;
int32 nodes_expanded;
}

```

10.7.2 Mandatory log/KPI set

For each move (and aggregated over games), the engine must log:

- Win-rate / Elo vs baseline at matched average time.
- Time histogram: easy-move fraction ($< 1s$) and hard-move fraction ($> 10s$), plus cap hits ($= T_{\max}$).
- Calibration: ECE of $\hat{P}_{\text{flip}}$ and a “flip-occurrence” confusion table.
- Scheduler health: stale-task discard rate, heap size, per-task wall time EMA.
- Blunder proxy: frequency that the top-1 root action changes late in search.

10.8 Reference implementation strategy

A minimal implementation can be done in Python for early experiments, but the above contracts are written to be implementable efficiently in C++ (with `pybind11` bindings) for consumer CPU deployment.

11 Implementation Notes (CPU-centric)

AVX2 and vectorization. All hot loops (selection score evaluation over live-set, backup updates, TT lookups) should be implemented in C++ with contiguous arrays (SoA layout) enabling SIMD. Neural inference may use ONNX Runtime or oneDNN-backed libraries; batching is beneficial even on CPU if latency constraints are respected.

GPU training. Self-play generation can run on GPU servers by batching inference; QUARTZ itself remains unchanged and can run with a GPU-backed `EvalBackend`. For fair comparisons, baseline and QUARTZ must use identical inference batching and TT/parallelism infrastructure.

12 Limitations and Open Issues

- **Belief misspecification:** VOC estimators rely on calibrated uncertainty; heavy-tail regimes require careful robust statistics.
- **Task staleness:** GVOC heap entries become stale under heavy parallel updates; lazy invalidation must be correct and cheap.
- **Tree-only curvature:** ORC/FRC computation adds overhead; it must be optional and carefully gated.
- **Hyperparameter minimization:** QUARTZ replaces many knobs with running medians/EMA, but some control parameters (time targets, live-set caps) remain.

13 Conclusion

QUARTZ formalizes a thermodynamic, physics-inspired but operationally grounded extension of neural MCTS. The core contribution is not a new value network or a larger search, but a principled *compute controller* that reallocates CPU time where it matters, activates rare-mode exploration only when warranted, and exposes a rigorous API to support spec-driven development and fair, reproducible comparisons.

References

- [1] S. Russell and E. Wefald. Principles of Metareasoning. *Artificial Intelligence*, 49(1–3):361–395, 1991. <https://www.sciencedirect.com/science/article/pii/000437029190015C>
- [2] D. Silver et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm (AlphaZero). *arXiv:1712.01815*, 2017. <https://arxiv.org/abs/1712.01815>
- [3] L. Kocsis and C. Szepesvári. Bandit Based Monte-Carlo Planning. In *ECML*, 2006. <https://ggp.stanford.edu/readings/uct.pdf>
- [4] R. Coulom. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search. In *CG 2006*. <https://www.remi-coulom.fr/CG2006/>
- [5] A. Couëtoux et al. Continuous Upper Confidence Trees. In *LION 2011*. https://link.springer.com/chapter/10.1007/978-3-642-25566-3_32
- [6] J. Skilling. Nested Sampling for General Bayesian Computation. *Bayesian Analysis*, 2006 (originally circulated 2004). <https://projecteuclid.org/journals/bayesian-analysis/volume-1/issue-4/Nested-sampling-for-general-Bayesian-computation/10.1214/06-BA127.pdf>
- [7] K. Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 2010. <https://www.fil.ion.ucl.ac.uk/~karl/NRN.pdf>
- [8] C. Guo et al. On Calibration of Modern Neural Networks. In *ICML*, 2017. <https://proceedings.mlr.press/v70/guo17a/guo17a.pdf>
- [9] A. Graves. Adaptive Computation Time for Recurrent Neural Networks. *arXiv:1603.08983*, 2016. <https://arxiv.org/abs/1603.08983>

- [10] A. Banino et al. PonderNet: Learning to Ponder. *arXiv:2107.05407*, 2021. <https://arxiv.org/abs/2107.05407>
- [11] G. Chaslot et al. Parallel Monte-Carlo Tree Search. In *Computers and Games*, 2008. <https://dke.maastrichtuniversity.nl/m.winands/documents/multithreadedMCTS2.pdf>
- [12] Y. Ollivier. Ricci curvature of Markov chains on metric spaces. *Journal of Functional Analysis*, 2009. <https://www.sciencedirect.com/science/article/pii/S002212360800493X>
- [13] M. Weber et al. Characterizing Complex Networks with Forman-Ricci Curvature and Associated Geometric Flows. *arXiv:1607.08654*, 2016. <https://arxiv.org/abs/1607.08654>